Faculdade de Engenharia da Universidade do Porto



Gestão do festival de música Evil Live (2025)

Submissão 2

Base de Dados 2025/26 - Licenciatura em Engenharia Informática e Computação

Grupo 0501 - Turma 2LEIC05

Estudantes & Autores

Gabriel Nunes Ribeiro de Magalhães up202405890@up.pt

Jorge Guilherme Freire da Cunha up202405961@up.pt

Pedro Afonso Seixas Pires up202405967@up.pt

Índice

Modelo Conceptual Refinado..... 3
Modelo Relacional..... 5
Versão inicial.....5
Integração de IA generativa.....7
Versão final.....8
Dependências funcionais e formas normais..... 9
Análise inicial.....9
Identificação das dependências funcionais para cada relação..... 9
Identificação das violações da 3NF e da Forma Normal de Boyce-Codd..... 10
Decomposição das relações que violam formas normais..... 10
Integração de IA generativa.....10
Versão final..... 11
Modelo Relacional (versão após normalização das dependências funcionais)..... 12
Criação da base de dados em SQLite..... 13
Versão inicial.....13
Integração de IA generativa.....13
Versão final.....14
População dos dados..... 15
Versão inicial.....15
Integração com IA generativa.....15
Versão final.....18
Modelo Conceptual Final..... 19
Modelo Relacional Final.....20
SQLite Final.....21
Análise crítica do desempenho da ferramenta de IA..... 22
Participação dos elementos do grupo..... 23

Modelo Conceptual Refinado

Após o feedback relativo à primeira submissão do trabalho, foram sugeridas algumas modificações ao modelo UML enviado, de modo a torná-lo o mais claro e completo possível.

De acordo com o feedback apresentado foram efetuadas as seguintes alterações:

1. O tempo de cada dia de festival passou a ser apresentado de forma normalizada, em que timestamps definem o início e fim de cada dia de festival.
2. Criou-se também um atributo tsCompra na classe Compra, permitindo análises de compras por janelas de tempo e sazonalidade intra dia.
3. O preço unitário de cada artigo tornou-se num atributo da classe de associação entre Compra e Artigo, que agora se passou a chamar CompraArtigo, permitindo flexibilidade do preço de um artigo no momento da compra (o valor de compra pode ser volátil ao longo do festival).
4. Foi adicionada uma restrição para o tipo de cada artigo existente (merch, comida ou bebida).
5. Inclusão de uma classe FluxoMonetário que permite registar todo o fluxo de receita (bilhetes, estabelecimentos comerciais e patrocinadores) e de despesa (staff, cachês das bandas), do ponto de vista do festival, de forma a facilitar a análise ao balanço financeiro final.

De seguida, numa análise do modelo conceptual final por parte do grupo, por questões de clareza, alterou-se o atributo da classe Artigo “custo” para “custoObtenção”, que representa o custo de obtenção do artigo para o vendedor (diferente do preço de venda). Para além disso, eliminou-se o atributo “númeroCC” da classe Staff e adicionaram-se os atributos “NIF” e “IBAN”, sendo que estes é que são realmente relevantes para se efetuar o pagamento ao Staff pelos seus serviços.

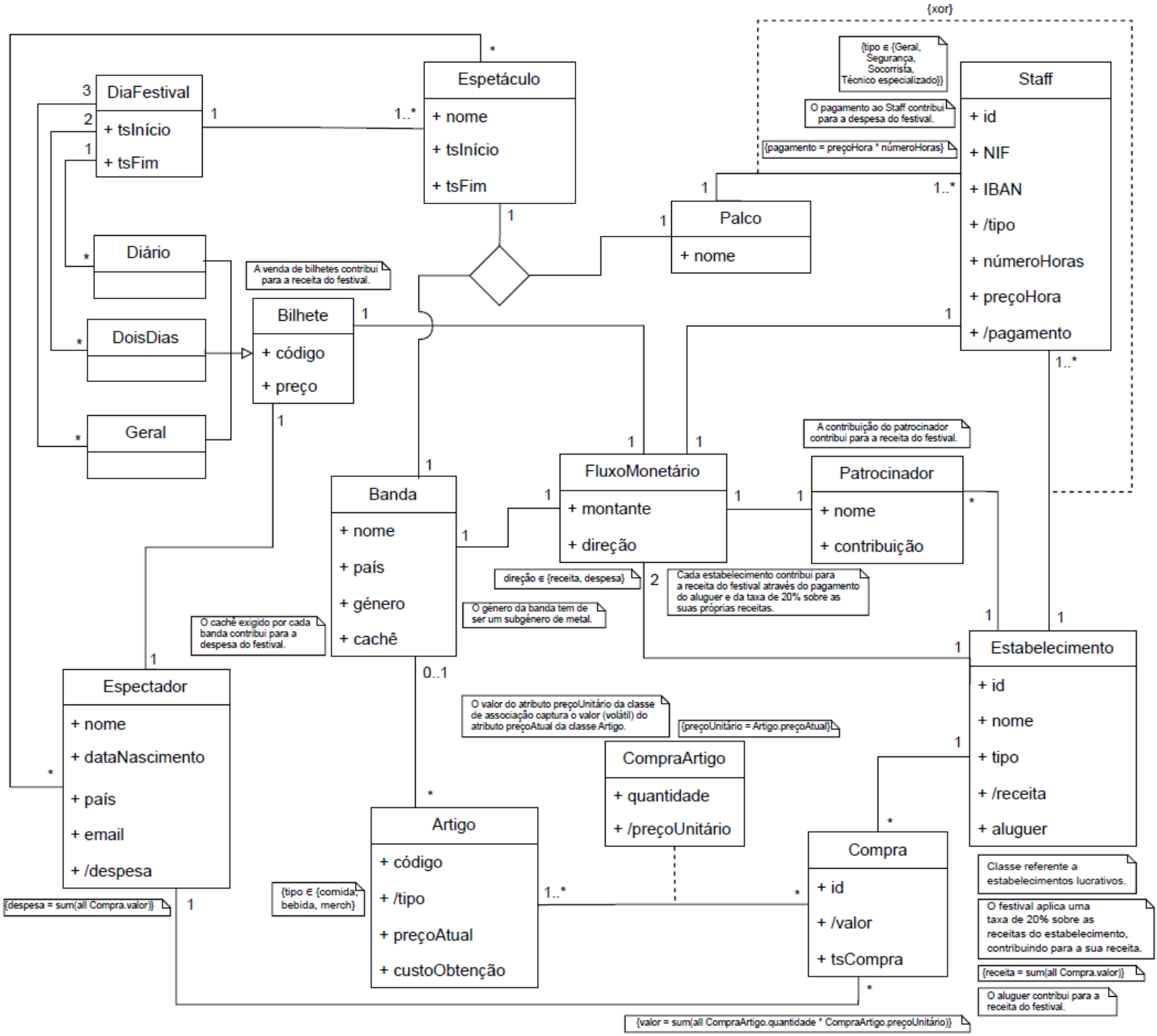


Figura 1 - Modelo conceptual refinado

Modelo Relacional

Versão inicial

Para as generalizações, optou-se pela abordagem Object-Oriented para modelar a hierarquia de bilhetes e E/R para os locais onde estariam distribuídos os membros do Staff do festival, generalização sugerida no feedback à submissão 1.

No primeiro caso, a generalização existente no diagrama UML é simultaneamente disjunta (cada bilhete pode ser apenas Diário, DoisDias ou Geral) e completa (um bilhete tem de ser necessariamente de um desses subtipos). Esta abordagem permite um número menor de relações (minimiza o espaço utilizado - permite remover a superclasse substituindo-a diretamente pelas classes especializadas), cumprindo a funcionalidade pretendida

No segundo caso, inicialmente o diagrama tinha uma restrição XOR que garantia que um funcionário do Staff apenas poderia estar alocado ou ao palco ou a um dos estabelecimentos do festival. Como sugerido pelo feedback à submissão 1, este aspeto é válido no modelo conceptual mas não no relacional dado que seria mais lógico e robusto utilizar uma superclasse Local tal que Palco e Estabelecimento fossem subtipos desta. Tal foi feito utilizando a abordagem E/R para generalizações.

DiaFestival (id, tsInício, tsFim)

Espetáculo (id, nome, tsInício, tsFim, DiaFestival_id→DiaFestival) *(os atributos tsInício e tsFim são chaves secundárias)*

Espectador (id, nome, dataNascimento, país, email, despesa)

EspetáculoEspectador (Espetáculo_id→Espetáculo, Espectador_id→Espectador)

Banda (id, nome, país, género, cachê)

Staff (id, NIF, IBAN, tipo, númeroHoras, preçoHora, pagamento, Local_id→Local) *(os atributos NIF e IBAN são chaves secundárias)*

Local(id, tipo)

Estabelecimento (id, Local_id→Local, nome, tipo, receita, aluguer)

Palco (id→Local, nome)

EspetáculoBandaPalco (Espetáculo_id→Espetáculo, Banda_id→Banda, Palco_id→Palco)

BilheteDiário (código, preço, Espectador_id→Espectador, DiaFestival_id→DiaFestival)

BilheteDoisDias (código, preço, Espectador_id→Espectador)

DiaFestivalBilheteDoisDias (DiaFestival_id→DiaFestival, BilheteDoisDias código→BilheteDoisDias)

BilheteGeral (código, preço, Espectador_id→Espectador)

DiaFestivalBilheteGeral (DiaFestival_id→DiaFestival, BilheteGeral_código → BilheteGeral)

Patrocinador (id, nome, contribuição, Estabelecimento_id→Estabelecimento)

Artigo (código, tipo, preçoAtual, custoObtenção, banda_id→Banda)

Compra (id, valor, tsCompra, Espectador_id→Espectador, Estabelecimento_id→Estabelecimento)

CompraArtigo (Artigo_código→Artigo, Compra_id→Compra, quantidade, preçoUnitário)

FluxoMonetário(id, montante, direção, Banda_id→Banda, Bilhete_id→Bilhete, Staff_id→Staff, Estabelecimento Id→Estabelecimento, Patrocinador_id→Patrocinador)

Integração de IA generativa

Fornecemos o modelo relacional à ferramenta de IA Perplexity para obter sugestões com vista a refiná-lo.

Prompt:

"*Com base no diagrama UML fornecido, analisa o modelo relacional também fornecido, avalia-o e apresenta possíveis sugestões de melhoria, de modo a produzir um produto final correto. [Explicação da introdução da relação Local, não presente no diagrama UML*] (Anexaram-se imagens do modelo UML e do modelo relacional)

Alteração:

Na relação EspetáculoBandaPalco, tornaram-se todas as chaves estrangeiras como chaves primárias da relação, uma vez que cada combinação de Espetáculo-Banda-Palco identifica unicamente um tuplo da relação.

Versão final

DiaFestival (id, tsInício, tsFim)

Espetáculo (id, nome, tsInício, tsFim, DiaFestival_id→DiaFestival) *(os atributos tsInício e tsFim são chaves secundárias)*

Espectador (id, nome, dataNascimento, país, email, despesa)

EspetáculoEspectador (Espetáculo_id→Espetáculo, Espectador_id→Espectador)

Banda (id, nome, país, género, cachê)

Staff (id, NIF, IBAN, tipo, númeroHoras, preçoHora, pagamento, Local_id→Local) *(os atributos NIF e IBAN são chaves secundárias)*

Local(id, tipo)

Estabelecimento (id, Local_id→Local, nome, tipo, receita, aluguer)

Palco (id→Local, nome)

EspetáculoBandaPalco (Espetáculo_id→Espetáculo, Banda_id→Banda, Palco_id→Palco)

BilheteDiário (código, preço, Espectador_id→Espectador, DiaFestival_id→DiaFestival)

BilheteDoisDias (código, preço, Espectador_id→Espectador)

DiaFestivalBilheteDoisDias (DiaFestival_id→DiaFestival, BilheteDoisDias_código→BilheteDoisDias)

BilheteGeral (código, preço, Espectador_id→Espectador)

DiaFestivalBilheteGeral (DiaFestival_id→DiaFestival, BilheteGeral_código→BilheteGeral)

Patrocinador (id, nome, contribuição, Estabelecimento_id→Estabelecimento)

Artigo (código, tipo, preçoAtual, custoObtenção, banda_id→Banda)

Compra (id, valor, tsCompra, Espectador_id→Espectador, Estabelecimento_id→Estabelecimento)

CompraArtigo (Artigo_código→Artigo, Compra_id→Compra, quantidade, preçoUnitário)

FluxoMonetário(id, montante,direção, Banda_id→Banda, Bilhete_id→Bilhete, Staff_id→Staff, Estabelecimento_id→Estabelecimento, Patrocinador_id→Patrocinador)

Dependências funcionais e formas normais

Análise inicial

Identificação das dependências funcionais para cada relação

Relação	Dependências Funcionais
Espetáculo	{id} → {nome, tsInício, tsFim, DiaFestival_id} {tsInício, tsFim} → {id, nome, DiaFestival_id}
Espectador	{id} → {nome, dataNascimento, país, email, despesa} {email} → {id, nome, dataNascimento, país, despesa}
Banda	{id} → {nome, país, género, cachê}
Staff	{id} → {NIF, IBAN, tipo, númeroHoras, preçoHora, pagamento, Local_id} {NIF} → {id, IBAN, tipo, númeroHoras, preçoHora, pagamento, Local_id} {IBAN} → {id, tipo, númeroHoras, preçoHora, pagamento, Local_id} {númeroHoras, preçoHora} → {pagamento}
Local	{id} → {tipo}
Estabelecimento	{id} → {nome, tipo, receita, aluguer, Local_id}
Palco	{id} → {nome}
DiaFestival	{id} → {tsInício, tsFim}
BilheteDiário	{código} → {preço, Espectador_id, DiaFestival_id}
BilheteDoisDias	{código} → {preço, Espectador_id}
BilheteGeral	{código} → {preço, Espectador_id}
Patrocinador	{id} → {nome, contribuição, Estabelecimento_id}
Artigo	{código} → {tipo, preçoAtual, custoObtenção, banda_id}
Compra	{id} → {valor, tsCompra, Espectador_id, Estabelecimento_id}
CompraArtigo	{Artigo_código, Compra_id} → {quantidade, preçoUnitário}
FluxoMonetário	{id} → {montante,direção, Banda_id, Bilhete_id, Staff_id, Estabelecimento_id, Patrocinador_id}

Relações resultantes de associações do modelo UML cujos os atributos são todos parte da primary key da relação (EspetáculoEspectador, EspetáculoBandaPalco, DiaFestivalBilheteDoisDias e DiaFestivalBilheteGeral) apenas têm uma dependência funcional trivial em que todos os atributos determinam todos os atributos.

Identificação das violações da 3NF e da Forma Normal de Boyce-Codd

De todas as dependências funcionais identificadas, apenas a seguinte viola a 3NF e a Forma Normal de Boyce-Codd:

{númeroHoras, preçoHora} → {pagamento} da relação Staff

…uma vez que {númeroHoras, preçoHora} não é uma superkey da relação Staff. Todas as outras dependências funcionais têm, do lado esquerdo da dependência, uma superkey das relações correspondentes, pelo que satisfazem imediatamente as duas formas normais.

Decomposição das relações que violam formas normais

Da decomposição da relação Staff obtiveram-se as seguintes novas relações:

Pagamento(<u>númeroHoras</u>, <u>preçoHora</u>, pagamento)
Staff(<u>id</u>,NIF,IBAN,tipo,númeroHoras→Pagamento.númeroHoras,preçoHora→Pagamento.preçoHora, Local_id→Local)

Integração de IA generativa

Fornecemos o conteúdo do ponto anterior à ferramenta de IA Perplexity para obter sugestões com vista a refiná-lo.

<u>Prompt:</u>

"*Com base no modelo relacional fornecido, sugere melhorias e efetua correções, se possível, à seguinte análise das suas dependências funcionais e formas normais: [dependências funcionais do ponto anterior*]" (Anexou-se a imagem do modelo relacional)

<u>Alteração:</u>

Após fornecido o prompt à ferramenta de IA, as alterações sugeridas pela ferramenta não se mostraram adequadas no contexto da base de dados, pelo que se optou por não as incluir e manter as dependências funcionais como descritas anteriormente.

Versão final

Relação	Dependências Funcionais
Espetáculo	{id} → {nome, tsInício, tsFim, DiaFestival_id} {tsInício, tsFim} → {id, nome, DiaFestival_id}
Espectador	{id} → {nome, dataNascimento, país, email, despesa} {email} → {id, nome, dataNascimento, país, despesa}
Banda	{id} → {nome, país, género, cachê}
Pagamento	{númeroHoras, preçoHora} → {pagamento}
Staff	{id} → {NIF, IBAN, tipo, númeroHoras, preçoHora, Local_id} {NIF} → {id, IBAN, tipo, númeroHoras, preçoHora, Local_id} {IBAN} → {id, tipo, númeroHoras, preçoHora, Local_id}
Local	{id} → {tipo}
Estabelecimento	{id} → {nome, tipo, receita, aluguer, Local_id}
Palco	{id} → {nome}
DiaFestival	{id} → {tsInício, tsFim}
BilheteDiário	{código} → {preço, Espectador_id, DiaFestival_id}
BilheteDoisDias	{código} → {preço, Espectador_id}
BilheteGeral	{código} → {preço, Espectador_id}
Patrocinador	{id} → {nome, contribuição, Estabelecimento_id}
Artigo	{código} → {tipo, preçoAtual, custoObtenção, banda_id}
Compra	{id} → {valor, tsCompra, Espectador_id, Estabelecimento_id}
CompraArtigo	{Artigo_código, Compra_id} → {quantidade, preçoUnitário}
FluxoMonetário	{id} → {montante,direção, Banda_id, Bilhete_id, Staff_id, Estabelecimento_id, Patrocinador_id}

Modelo Relacional (versão após normalização das dependências funcionais)

DiaFestival (id, tsInício, tsFim)

Espetáculo (id, nome, tsInício, tsFim, DiaFestival_id→DiaFestival) *(os atributos tsInício e tsFim são chaves secundárias)*

Espectador (id, nome, dataNascimento, país, email, despesa)

EspetáculoEspectador (Espetáculo_id→Espetáculo, Espectador_id→Espectador)

Banda (id, nome, país, género, cachê)

Pagamento(númeroHoras, preçoHora, pagamento)

Staff(id,NIF,IBAN,tipo,númeroHoras→Pagamento.númeroHoras, preçoHora→Pagamento.preçoHora, Local_id→Local) *(os atributos NIF e IBAN são chaves secundárias)*

Local(id, tipo)

Estabelecimento (id, Local_id→Local, nome, tipo, receita, aluguer)

Palco (id→Local, nome)

EspetáculoBandaPalco (Espetáculo_id→Espetáculo, Banda_id→Banda, Palco_id→Palco)

BilheteDiário (código, preço, Espectador_id→Espectador, DiaFestival_id→DiaFestival)

BilheteDoisDias (código, preço, Espectador_id→Espectador)

DiaDoBilheteDoisDias (DiaFestival_id→DiaFestival, BilheteDoisDias código→BilheteDoisDias)

BilheteGeral (código, preço, Espectador_id→Espectador)

DiaDoBilheteGeral (DiaFestival_id → DiaFestival, BilheteGeral_código → BilheteGeral)

Patrocinador (id, nome, contribuição, Estabelecimento_id→Estabelecimento)

Artigo (código, tipo, preçoAtual, custoObtenção, Banda_id→Banda)

Compra (id, valor, tsCompra, Espectador_id→Espectador, Estabelecimento_id→Estabelecimento)

CompraArtigo (Artigo_código→Artigo, Compra_id→Compra, quantidade, preçoUnitário)

FluxoMonetário(id, montante,direção, Banda_id→Banda, Bilhete_id→Bilhete, Staff_id→Staff, Estabelecimento_id→Estabelecimento, Patrocinador_id→Patrocinador)

Criação da base de dados em SQLite

Versão inicial

Foi criado o ficheiro create1.sql, tendo em conta o modelo relacional anterior.

Integração de IA generativa

Fornecemos o ficheiro create1.sql e o modelo relacional à ferramenta de IA Perplexity para obter sugestões com vista a refiná-lo.

Prompt:

"Com base no ficheiro create1.sql anexado (que por sua vez segue o modelo relacional também anexado), sugere melhorias e efetua correções ao código SQLite do ficheiro .sql, se possível. Não implementes triggers nem uses AUTOINCREMENT." (Anexou-se o ficheiro create1.sql e a imagem do modelo relacional)

Alterações:

A ferramenta de IA identificou os seguintes problemas:

1) "Pagamento - Chave Primária Inadequada: (numeroHoras, precoHora) não identifica tuplos únicos; deveria ter coluna id";
2) Erros de sintaxe.

Após análise do feedback recebido da IA, concluiu-se, então, que staffs com o mesmo número de horas e preço por hora iriam referenciar o mesmo tuplo na relação Pagamento, e desta forma perderia-se unicidade nos registos de pagamento. Para além disso, haveria informação duplicada dado que os atributos númeroHoras e preçoHora estariam ambos presentes nas relações Staff, como foreign keys, e Pagamento. Para simplificar este processo, decidiu-se **eliminar a relação Pagamento**, bem como as FK númeroHoras e preçoHora de Staff e criar um único atributo nesta relação, vencimento, que indica o vencimento de cada Staff ao longo do festival. A normalização efetuada para a forma normal de Boyce-Codd anulou-se tendo em conta esta alteração. Contudo, considerando a praticidade global da base de dados, concluiu-se que esta foi a melhor solução.

Nota: Por causa destas alterações, os ficheiros populate1.sql e populate2.sql, que serão explicados na secção posterior, não funcionam com a base de dados definida pelo ficheiro create1.sql.

Para além disso, corrigiram-se também os erros de sintaxe sinalizados.

Destas alterações, surgiu o ficheiro create2.sql.

Versão final

Para além das alterações feitas no passo de integração de IA generativa, adicionaram-se comentários ao ficheiro create2.sql que descrevem os triggers necessários para manter a coerência de dados da base de dados, bem como a adição de novas restrições (CHECK constraints) que se consideraram importantes para o realismo dos dados inseridos.

Também foram simplificados nomes das tabelas de associações, para torná-las mais legíveis: ‘EspetaculoEspectador’ passou a chamar-se ‘Assiste’, ‘EspetaculoBandaPalco’ passou a chamar-se ‘Concerto’, as relações entre ‘DiaFestival’ e ‘BilheteDoisDias’ (‘DiaFestivalBilheteDoisDias’) ou ‘BilheteGeral’ (‘DiaFestivalBilheteGeral’) para ‘DiaDoBilheteDoisDias’ e ‘DiaDoBilheteGeral’ e a relação ‘CompraArtigo’ para ‘ArtigoNumaCompra’.

População dos dados

Versão inicial

Durante a população da base de dados efetuaram-se mudanças ao ficheiro create2.sql, refinado pelo modelo de IA referido acima: adicionaram-se novas restrições e identificaram-se mais triggers necessários à manutenção da integridade referencial da base de dados.

Inicialmente populou-se a base de dados com poucos dados, criando o ficheiro populate1.sql, mantendo a consistência dos dados tendo em conta os triggers previamente identificados, ainda que não implementados. Isto fará com que, no passo seguinte, a ferramenta de IA mantenha a mesma coerência dos dados, por análise dos mesmos até então inseridos.

Integração com IA generativa

Para facilitar a compreensão das restrições e dos triggers necessários, comentou-se cada tabela no ficheiro populate1.sql com instruções necessárias para a inserção dos dados com ajuda do modelo de IA, mantendo a consistência dos dados e adicionando realismo à base de dados.

Por exemplo, na tabela Espectador, identificaram-se características que seriam necessárias à consistência e realismo dos dados na base de dados:

*/*Cria mais 200 espectadores, com diferentes nomes, datas de nascimento, países e emails. Mas a sua despesa tem que ter em conta a tabela Compra: a despesa de Espectador tem que ser a soma de cada valor em Compra com o seu Espectador_id correspondente. A despesa do espectador deve rondar os 40€ em média, mas podem haver espectadores que não gastaram nada (raro). Esta despesa do espectador deve corresponder com as compras (tabela Compra) feitas por esses espectador. O ano de nascimento do espectador deve rondar o ano 1995, em média (podem haver mais velhos e mais novos). O país maioritário deve ser Portugal, mas deves inserir espectadores de outros países.*/*

Os prompts enviados contêm, de forma estruturada, os pontos de melhoria identificados após cada versão do ficheiro populate2.sql gerada pela ferramenta de IA.

Nota: Para não criar uma base de dados excessivamente grande, limitamos o número de espectadores a 100, que é uma amostra suficientemente representativa e que já permite a análise de informação considerável.

Prompt 1:

“Analisa cuidadosamente ambos os ficheiros enviados, relativos a uma base de dados para gestão do festival de música português Evil Live, edição 2025/26. Tendo em atenção todas as restrições e comentários presentes no ficheiro create2.sql e as instruções para inserção de tuplos em populate1.sql, adiciona tuplos às tabelas. Das informações que conseguires retirar acerca desta edição do festival, tenta inserir dados o mais realista que conseguires. Mas se não conseguires obter certas informações, insere dados fictícios. Popula com extremo cuidado para respeitar todas as restrições e comentários dos dois ficheiros.” (Anexaram-se os ficheiros populate1.sql e create2.sql).

Prompt 2: (no seguimento do prompt anterior):

“Altera o ficheiro enviado tendo as conta as seguintes alterações:

1) Na tabela Espectador, não insiras nomes nem e-mails com plicas, para evitar erros com strings.

2) Na tabela Espectador, coloca os nomes dos países em Português.

3) Insere mais bilhetes nas tabelas correspondentes a bilhete. O número de bilhetes existentes deve ser igual ao número de espectadores existentes. Tem em conta todas as restrições e comentários dos ficheiros create2.sql e populate1.sql para fazeres isto (uma das restrições: o id de um BilheteDiario não pode ser um id de BilheteDoisDias e BilheteGeral,... não podem haver ids comuns entre os tipos de bilhete).

4) Insere mais tuplos na tabela EspetaculoEspectador, de modo a ser compatível com os tuplos na tabela Espectador (de momento, existem mais tuplos de Espectador do que de EspetaculoEspectador, o que não faz sentido, pois todos os Espectadores normalmente vêem pelo menos um espetáculo). O tuplo inserido deve também ter em conta o bilhete que o espectador comprou, pois este apenas pode ver espetáculos que aconteceram em dias cujo seu bilhete lhe concede acesso. Deve haver muitos mais tuplos de EspetaculoEspectador do que de Espectador.

5) Na tabela FluxoMonetario, substitui os None's por NULL's.

6) Tem a certeza que o atributo receita de cada Estabelecimento corresponde à soma de todas as compras lá realizadas.

7) Para cada bilhete criado no passo 3), cria um tuplo correspondente de 'receita' na tabela FluxoMonetario.

8) Tem em conta que os tuplos em FluxoMonetario referentes a Estabelecimentos e à sua receita, têm o atributo montante igual a 20% da receita desse estabelecimento (esta

restrição e outras estão presentes nos ficheiros create2.sql e populate1.sql, os quais deves analisar cuidadosamente).” (Anexaram-se os ficheiros populate2.sql e create2.sql).

Prompt 3: (no seguimento do prompt anterior):
“Altera o ficheiro populate2.sql enviado tendo as conta as seguintes alterações:
1) Reduz o número de espectadores mais ou menos para 100 e o número de compras mais ou menos para 200 (em média duas compras por espectador, sendo que podem haver espectadores que não fizeram nenhuma compra, poucos).
2) Tendo em conta a alteração do número de espectadores anterior alteras as tabelas BilheteDiario, BilheteDoisDias e BilheteGeral, sendo que cada espectador apenas pode ter um destes bilhetes, o que implica verificações nas foreign keys destas três tabelas (um id de um espectador numa destas tabelas não pode estar nas outras duas) (ler restrições e triggers em create2.sql). Altera também a tabela Assiste, para apenas conter ids de espectadores válidos e refletindo a realidade de que um espectador apenas tenha visto espetáculos que aconteceram em dias para os quais o seu bilhete dá acesso. Normalmente, os espectadores que foram a um dia do espetáculo viram a maioria dos espetáculos que aconteceram nesse dia. Por isso, espectadores com BilheteGeral podem ter visto todos espetáculos do festival.
3) A tabela ArtigoNumaCompra apenas tem dois tuplos, quando na verdade deveria ter muitos mais, refletindo a quantidade e o preço de cada artigo comprado em cada compra (as compras podem tanto ser de apenas um artigo, como de vários artigos: se for de apenas um artigo, deve haver apenas um tuplo na tabela ArtigoNumaCompra, se não, devem haver mais, consoante o número de artigos presentes numa compra). A soma dos preços dos artigos associados a uma compra devem corresponder ao valor da compra e os valores das compras associados a um espectador devem corresponder à sua despesa (restrições presentes em triggers no ficheiro create2.sql, revê esses triggers).
4) Adiciona aos tuplos de Compra o atributo tsCompra (como presente no ficheiro create2.sql), que indica o instante em que a compra foi feita, tendo em conta o seguinte: O atributo tsCompra tem de estar compreendido entre os tsInicio e tsFim de um dia do festival ao qual o espectador que realizou a compra (atributo Espectador_id) tenha acesso (verificar qual é o bilhete desse espectador e os dias associados a esse bilhete).” (Anexaram-se os ficheiros populate2.sql e create2.sql).

Prompt 4:

“No ficheiro anterior, populate2.sql, encontramos alguns pontos de melhoria:

1) Os timestamps das compras têm de ser consistentes com a realidade: o atributo tsCompra tem de estar compreendido entre os tsInicio e tsFim de um dia do festival ao qual o espectador que realizou a compra (visível através do atributo Espectador_id) tenha acesso, e isto para qualquer espectador na base de dados. Os timestamps das compras têm valores no campo das horas (no formato YYYYMMDDhhmmss) que não fazem sentido, como 58, 30,...

2) Reduzir os preços de aluguer de cada estabelecimento para um valor razoável uma vez que o valor anterior era demasiado elevado, quase nenhum dos estabelecimentos teria tido lucro. Mantém os estabelecimentos existentes, apenas efetua esta mudança;

3) Acrescentar tuplos, do tipo ‘receita’, à tabela FluxoMonetário que refletissem o montante associado ao aluguer, pago por cada estabelecimento, tal como foram modificados no ponto 2). Mantém aquilo que estava correto anteriormente;

4) Na relação Staff existiam estabelecimentos sem qualquer staff, por exemplo os estabelecimentos de venda poderiam ter staff geral, o palco deveria ter mais seguranças - por exemplo para 100 espectadores teríamos 2 seguranças no palco. O número total de staff deve ser consistente (rácio adequado) com o número de espectadores, o número de estabelecimentos e o facto de existir um palco.

5) Na tabela Espectador, altera o intervalo de datas de nascimento para [1960, 2010] e altera os seus nomes para serem consistentes com o seu país (não devem haver espectadores com nomes portugueses que são de outro país). Ao fazeres isto, não adiciones nem modifiques os outros atributos dos tuplos desta tabela. Apenas modifica os atributos relativos ao nome, data de nascimento e altera também os e-mails dos quais modificaste os nomes, para refletirem o novo nome, Não alteres o id nem a despesa! Isto é muito importante.” (Anexaram-se os ficheiros populate2.sql e create2.sql).

A ferramenta de IA usada, ao longo dos sucessivos prompts, apenas adicionou conteúdo às sucessivas versões do ficheiro populate2.sql, sempre respeitando as orientações fornecidas. Assim, as versões do ficheiro populate2.sql foram progressivamente melhorando, produzindo informação coerente na base de dados e culminando na versão final do ficheiro.

Versão final

A versão final do ficheiro populate2.sql, após a integração de IA generativa, contém instruções que permitem produzir uma base de dados coesa e representativa (em parte) do festival Evil Live, edição 2025.

Modelo Conceptual Final

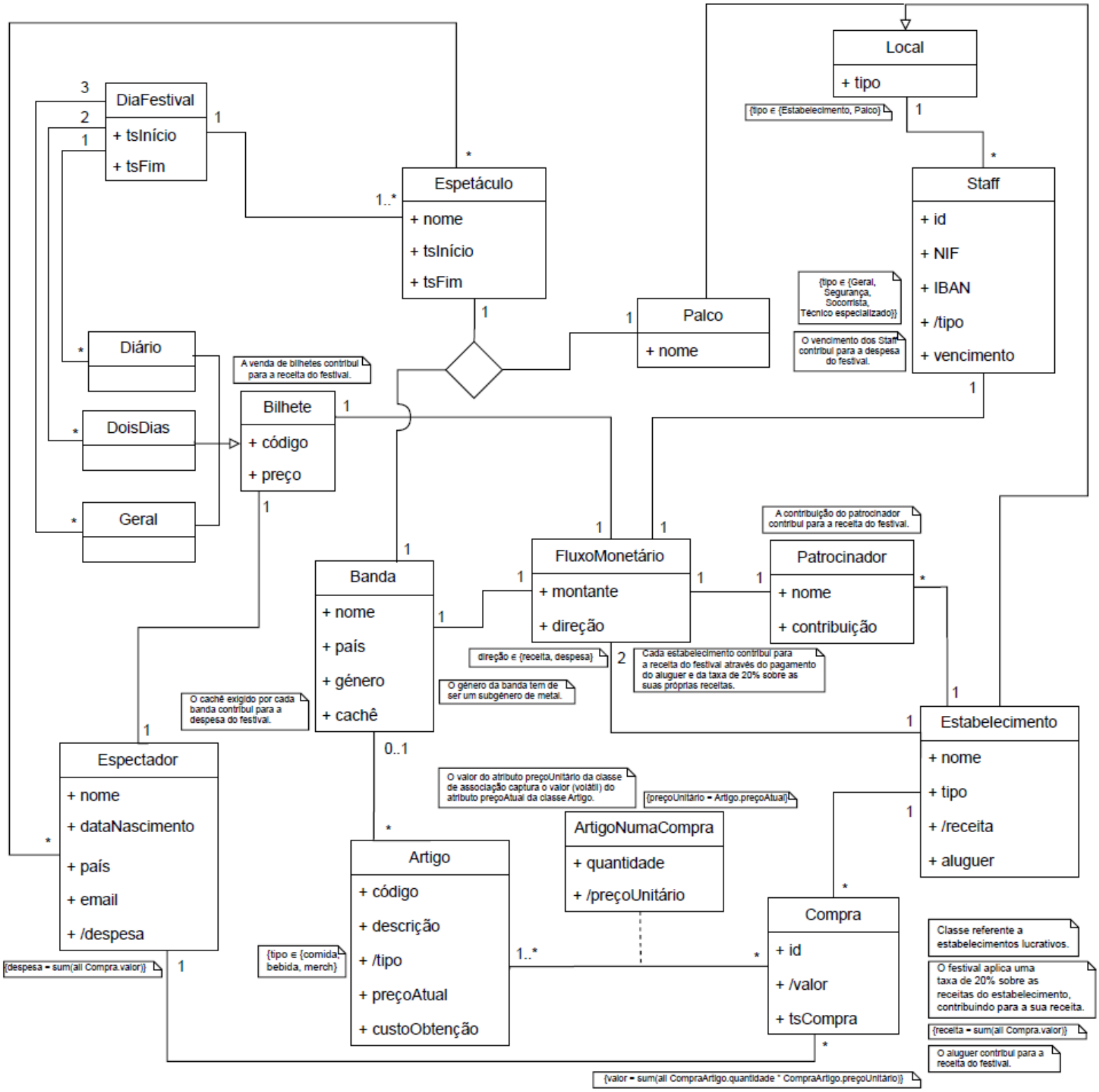


Figura 2 - Modelo conceptual final

Modelo Relacional Final

DiaFestival (id, tsInício, tsFim)

Espetáculo (id, nome, tsInicio, tsFim, DiaFestival_id→DiaFestival)

Espectador (id, nome, dataNascimento, país, email, despesa)

Assiste (Espetáculo_id→Espetáculo, Espectador_id→Espectador)

Banda (id, nome, país, género, cachê)

Staff(id,NIF,IBAN,tipo,vencimento, Local_id→Local)

Local(id, tipo)

Estabelecimento (Local_id→Local, nome, tipo, receita, aluguer)

Palco (Local_id→Local, nome)

Concerto (Espetáculo_id→Espetáculo, Banda_id→Banda, Palco_id→Palco)

BilheteDiário (código, preço, Espectador_id→Espectador, DiaFestival_id→DiaFestival)

BilheteDoisDias (código, preço, Espectador_id→Espectador)

DiaDoBilheteDoisDias (DiaFestival_id→DiaFestival, BilheteDoisDias código→BilheteDoisDias)

BilheteGeral (código, preço, Espectador_id→Espectador)

DiaDoBilheteGeral (DiaFestival_id → DiaFestival, BilheteGeral_código → BilheteGeral)

Patrocinador (id, nome, contribuição, Estabelecimento_id→Estabelecimento)

Artigo (código, descrição, tipo, preçoAtual, custoObtenção, Banda_id→Banda)

Compra (id, valor, tsCompra, Espectador_id→Espectador, Estabelecimento_id→Estabelecimento)

ArtigoNumaCompra(Artigo_código→Artigo, Compra_id→Compra, quantidade, preçoUnitário)

FluxoMonetário(id,montante,direção,Banda_id→Banda,BilheteGeral_codigo→BilheteGeral, BilheteDoisDias_codigo→BilhetesDoisDias,BilheteDiario_codigo→BilheteDiario, Staff_id→Staff, Estabelecimento_id→Estabelecimento, Patrocinador_id→Patrocinador)

SQLite Final

Após sucessivas iterações, já apresentadas anteriormente, envolvendo modelação relacional, passagem para o SQLite e população da base de dados, obtiveram-se os ficheiros create2.sql e populate2.sql. Estes permitem simular o comportamento desta base de dados num contexto próximo a uma situação real, possibilitando a extração e análise de informação relativa ao festival Evil Live, edição 2025.

Deste modo é possível compreender o funcionamento logístico do festival e tirar conclusões importantes para a sua melhoria e otimização do mesmo ao longo dos anos, correspondendo exatamente ao objetivo estipulado na primeira submissão deste trabalho.

Análise crítica do desempenho da ferramenta de IA

A ferramenta de IA utilizada ao longo de todo o trabalho foi o PerplexityAI especificamente através da funcionalidade Labs, que permite ao modelo pensar e refletir mais tempo antes de fornecer uma resposta.

Este modelo revelou-se bastante útil mas a sua intervenção foi mais notória no processo de População de Dados, em que a IA foi capaz de cumprir as múltiplas restrições impostas pelo grupo para a população dos dados com grande fiabilidade e relativa velocidade. Para além disso, os dados por esta gerados foram bastante realistas, ainda que nos prompts isso tenha sido expressamente pedido e alguns dos dados foram manualmente introduzidos. Dado que o festival que se decidiu modelar não revela publicamente os dados (por motivos óbvios de confidencialidade), a IA provavelmente seria a única solução para este problema. Tentar reproduzir manualmente tudo o que a IA fez seria muito exaustivo ou praticamente impossível em tempo útil, dada a dimensão do próprio ficheiro e as múltiplas e complexas verificações e restrições ao nível de triggers.

Contudo, nem sempre o desempenho da ferramenta foi o esperado e diversas vezes eram visíveis algumas alucinações, fenómeno transversal a qualquer modelo - priorizam responder ao utilizador face à garantia de exatidão. Podem-se destacar algumas incorreções em sugestões feitas em relação ao modelo conceptual e na tentativa de refinação das dependências funcionais, em que certas respostas foram completamente descartadas por não se mostrarem adequadas.

De modo a extrair a melhor performance do modelo, para cada prompt fizeram-se diversas iterações que culminaram em refinamentos sucessivos que, como referido anteriormente, nem sempre foram aceites.

Em suma, considerou-se o desempenho do modelo como positivo porque facilitou certas partes mais trabalhosas e repetitivas e identificou problemas de forma rápida, sugerindo mudanças adequadas, mostrando excelente “raciocínio”.

Porém, a correta utilização destas ferramentas requer uma análise crítica rigorosa das respostas e estruturação correta dos prompts. O utilizador é, em última análise, o responsável por tudo o que assume como correto.

Participação dos elementos do grupo

Todos os elementos do grupo contribuíram igualmente e estiveram envolvidos em todas as partes do trabalho, pelo que a contribuição de cada um foi um terço do mesmo.